C++ QUICK REFERENCE / C++ CHEATSHEET

Based on Phillip M. Duxbury's C++ Cheatsheet and Morten Nobel-Jørgensen edits

Preprocessor

```
// Comment to end of line
/* Multi-line comment */
#include <stdio.h> // Insert standard header file
#include "myfile.h" // Insert file in current directory
#define X some text // Replace X with some text
#define F(a,b) a+b // Replace F(1,2) with 1+2
#define X \
 some text // Multiline definition
#undef X // Remove definition
#if defined(X) // Conditional compilation (#ifdef X)
#else // Optional (#ifndef X or #if !defined(X))
#endif // Required after #if, #ifdef
```

Literals

```
255, 0377, 0xff // Integers (decimal, octal, hex)
2147483647L, 0x7fffffffl // Long (32-bit) integers
123.0, 1.23e2 // double (real) numbers
'a', '\141', '\x61' // Character (literal, octal, hex)
'\n', '\\', '\'', '\"' // Newline, backslash, single quote,
  double quote
"string\n" // Array of characters ending with newline and \0
"hello" "world" // Concatenated strings
true, false // bool constants 1 and 0
nullptr // Pointer type with the address of 0
```

Declarations

```
int x; // Declare x to be an integer (value undefined)
int x=255; // Declare and initialize x to 255
short s; long l; // Usually 16 or 32 bit integer (int may be
  either)
char c='a'; // Usually 8 bit character
unsigned char u=255;
signed char s=-1; // char might be either
unsigned long x = 0xffffffffL; // short, int, long are signed
float f; double d; // Single or double precision real (never
  unsigned)
bool b=true; // true or false, may also use int (1 or 0)
int a, b, c; // Multiple declarations
int a[10]; // Array of 10 ints (a[0] through a[9])
int a[]={0,1,2}; // Initialized array
int a[2][2]={ {1,2},{4,5} }; // Array of array of ints
char s[]="hello"; // String (6 elements including '\0')
std::string s = "Hello" // Creates string object with value "
  Hello"
std::string s = R"(Hello\nWorld)"; // Creates string object with
   value "Hello\nWorld"
int* p; // p is a pointer to int
char* s="hello"; // s points to unnamed array containing "hello
  "
void* p=nullptr; // Address of untyped memory
int& r=x; // r is a reference to (alias of) int x
enum weekend {SAT,SUN}; // weekend is a type with values SAT and
   SUN
enum weekend day; // day is a variable of type weekend
enum weekend{SAT=0,SUN=1}; // Explicit representation as int
enum {SAT,SUN} day; // Anonymous enum
enum class Color {Red,Blue};// Strict enum type
Color x = Color::Red; // Assign Color x to red
typedef char* String; // String s; means char* s;
const int c=3; // Constants must be initialized
const int* p=a; // Contents of p are constant
int* const p=a; // p (not contents) is constant
const int* const p=a; // Both p and contents constant
const int& cr=x; // cr cannot assign to change x
int8_t,uint8_t,int16_t,uint16_t,int32_t,uint32_t,int64_t,uint64_t
   // Fixed length types
auto it = m.begin(); // Declares it to the result of m.begin()
auto const param = config["param"]; // Declares const result
auto& s = singleton::instance(); // Declares reference to result
```

Storage Classes

```
int x; // Auto (memory exists only while in scope)
static int x; // Global lifetime even if local scope
extern int x; // Declared elsewhere
```

Statements

```
x=y; // Every expression is a statement
int x; // Declarations are statements
; // Empty statement
{ int x; } // Block scope
if (x) a; else if (y) b; else c;
while (x) a;
for (x; y; z) a;
for (x : y) a; // Range-based for
do a; while (x);
switch (x){ case X1:a; case X2:b; default:c; }
break; continue; return x;
try { a; } catch (T t) { b; } catch (...) { c; }
```

Expressions

```
T::X, N::X, ::X // Name lookup
t.x, p->x, a[i], f(x,y), T(x,y)
x++, x-- // Postfix increment/decrement
++x, --x // Prefix increment/decrement
typeid(x), typeid(T)
dynamic_cast<T>(x), static_cast<T>(x), reinterpret_cast<T>(x),
  const_cast<T>(x)
sizeof x, sizeof(T)
~x, !x, -x, +x, &x, *p
new T, new T(x,y), new T[x]
delete p, delete[] p
(T) x // C-style cast
x * y, x / y, x % y
x + y, x - y, x << y, x >> y
x < y, x <= y, x > y, x >= y
x & y, x ^ y, x | y
x && y, x || y
x = y, x += y, x -= y, x *= y, x /= y, x <<= y, x >>= y, x &= y,
  x |= y, x ^= y
x ? y : z
throw x
x , y // evaluates x then y
```

Classes

```
class T {
private: int x;
protected: int y;
public:
    void f(); void g() { return; }
    void h() const;
    int operator+(int y);
    int operator-();
    T():x(1){} // Constructor
    T(const T& t):x(t.x){} // Copy constructor
    T& operator=(const T& t){ x=t.x; return *this; } //
      Assignment
    ~T(); // Destructor
    explicit T(int a); // explicit constructor
    T(float x):T((int)x){} // Delegate constructor
    operator int() const {return x;}
    friend void i();
    friend class U;
    static int y;
    static void l();
    class Z {};
    typedef int V;
};
struct T{ virtual void i(); virtual void g()=0; };
class U: public T { void g(int) override; };
class V: private T {};
class W: public T, public U {};
class X: public virtual T {};
```

Templates

```
template<class T> T f(T t);
template<class T> class X{ X(T t); };
template<class T> X<T>::X(T t) {}
X<int> x(3);
template<class T, class U=T, int n=0> class Y{};
```

Namespaces

```
namespace N { class T {}; }
N::T t;
using namespace N;
```

Memory Management

```
#include <memory>
shared_ptr<int> x; // Empty shared_ptr
x = make_shared<int>(12); // Allocate on heap
shared_ptr<int> y = x; // Shared ownership
cout << *y; // Dereference
y.reset(); // Remove ownership
weak_ptr<int> w = y; // Weak reference
if(shared_ptr<int> s = w.lock()){ cout << *s; }
unique_ptr<int> z; z = make_unique<int>(16);
unique_ptr<int> q; q = move(z);
if(z==nullptr) cout << "Z null";
cout << *q;
```

cmath

```
#include <cmath>
sin(x); cos(x); tan(x);
asin(x); acos(x); atan(x);
atan2(y,x);
sinh(x); cosh(x); tanh(x);
exp(x); log(x); log10(x);
pow(x,y); sqrt(x);
```

cassert

```
#include <cassert>
assert(e); // Abort if e is false
#define NDEBUG // Disable asserts
```

iostream

```
#include <iostream>
cin >> x >> y; // Read input
cout << "x=" << 3 << endl; // Write output
cerr << x << y << flush; // stderr
c = cin.get(); // getchar
cin.get(c);
cin.getline(s,n,'\n');
```

fstream

```
#include <fstream>
ifstream f1("filename"); if(f1) f1 >> x;
f1.get(s); f1.getline(s,n);
ofstream f2("filename"); if(f2) f2 << x;
```

string

```
#include <string>
string s1, s2="hello";
s1 += s2 + ' ' + "world";
s1 == "hello world";
s1[0]; s1.substr(m,n); s1.c_str();
s1 = to_string(12.05);
getline(cin,s);
```

vector / deque

```
#include <vector>
vector<int> a(10); vector<int> b{1,2,3};
a.push_back(3); a.back()=4; a.pop_back();
a.front(); a.at(20)=1; // safe vs unsafe access
for(int& p:a) p=0; // C++11
for(vector<int>::iterator p=a.begin(); p!=a.end(); ++p) *p=0; /
   C++03
vector<int> b(a.begin(), a.end());
vector<T> c(n,x); T d[10]; vector<T> e(d,d+10);

#include <deque>
deque<int> d;
d.push_front(x); d.pop_front();
```

pair / map / set

```
#include <utility>
pair<string,int> a("hello",3);
a.first; a.second;

#include <map>
map<string,int> a; a["hello"]=3;
for(auto &p:a) cout << p.first << p.second;

#include <unordered_map>
unordered_map<string,int> a; a["hello"]=3;

#include <set>
set<int> s; s.insert(123);
if(s.find(123) != s.end()) s.erase(123);

#include <unordered_set>
unordered_set<int> s; s.insert(123);
if(s.find(123) != s.end()) s.erase(123);
```